

Spring Boot JDBC — Data Access, Connection Pooling & Transactions

Chapter 4 — Pro Spring Boot 4 (4th ed) · `chapters/chapter-04-jdbc/customer`

Spring Boot 4.0.6

Spring Framework 7.0.7

spring-boot-starter-jdbc

HikariCP

H2 (dev) / PostgreSQL (prod)

JdbcTemplate + JdbcClient

Contents

1. [What this chapter covers](#)
2. [Code walkthrough](#)
3. [Mermaid diagrams](#)
4. [Live run & endpoint evidence](#)
5. [Session context](#)

1 · What this chapter covers

Chapter 4 replaces the in-memory map from earlier chapters with a real relational store, and shows two ways to talk to it:

- **DataSource + connection pooling** — Spring Boot autoconfigures a `HikariCP` `DataSource` from `spring.datasource.*`.
- **Schema initialization** — `schema.sql` runs on startup to create the `customer` table.
- **Classic data access** — `JdbcTemplate` with a hand-written `RowMapper` and positional `?` parameters.
- **Modern fluent data access** — `JdbcClient` with named `:param` binding and automatic row mapping.
- **Environment profiles** — H2 in-memory by default, PostgreSQL under the `prod` profile.

2 · Code walkthrough

Two repositories, one interface

CustomerRepository.java · JdbcClientCustomerRepository.java · Repository.java

Both classes implement the same `Repository<Customer, UUID>` contract but use different APIs:

- `CustomerRepository` uses `JdbcTemplate` — `query("SELECT * FROM customer WHERE id = ?", rowMapper, id)` with positional `?` placeholders and an explicit `RowMapper`.
- `JdbcClientCustomerRepository` uses the fluent `JdbcClient` — `.sql("... WHERE id = :id").param("id", id).query(Customer.class).optional()` with named parameters and automatic record mapping.

Only the `JdbcTemplate` variant is wired: `CustomerConfiguration` declares `@Bean CustomerRepository customerRepository(JdbcTemplate)`. The `JdbcClient` class has no `@Repository` /bean registration, so it is a parallel demonstration (and test target), not the live bean.

RowMapper → immutable record

CustomerRowMapper.java

Maps a JDBC `ResultSet` row to the `Customer` record, converting the `id` column with `UUID.fromString`. This is the classic Spring JDBC mapping contract.

DataSource, pool & schema init

application.properties · application-prod.properties · schema.sql

Default profile points at `jdbc:h2:mem:testdb`; HikariCP pools connections lazily on first use. `schema.sql` creates `customer` with a `UUID PRIMARY KEY` and `email VARCHAR(255) NOT NULL UNIQUE`. The `prod` profile swaps to `jdbc:postgresql://localhost:5432/crm` and forces `spring.sql.init.mode=always`.

Upsert-style `save()`

CustomerRepository.java:29-57

Because records are immutable, `save()` generates a `UUID` when the id is null and inserts; if the id exists it updates, otherwise it inserts the predefined id. Note the read-then-write is not wrapped in a transaction (see observations).

Controller & error handling

CustomerController.java · GlobalExceptionHandler.java

The web layer is unchanged from Chapter 3: `@Valid` + `ProblemDetail` for request validation. However, the advice only maps validation and not-found errors — database-level failures (e.g. the `UNIQUE` email constraint) fall through to Spring's default error response.

3 · Mermaid diagrams

Rendered from Mermaid and embedded as self-contained images (they also print in the PDF export). Expand each for the re-renderable source.

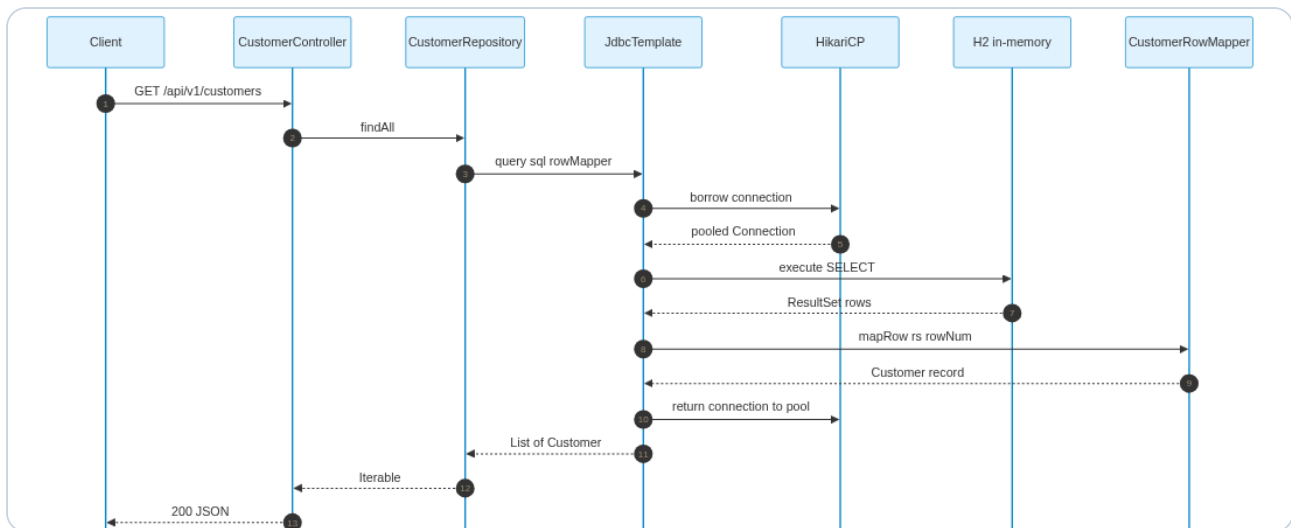


Fig 1 — Data-access lifecycle. A `GET` flows Controller → Repository → `JdbcTemplate`, which borrows a connection from `HikariCP`, runs the SQL against H2, maps each row via `CustomerRowMapper`, then returns the connection to the pool.

[Mermaid source](#)

```

%%{init: {'theme':'base','themeVariables':{'primaryColor':'#e0f2fe','primaryBorderColor':'#0284c7','line
sequenceDiagram
    participant CL as Client
    participant CT as CustomerController
    participant REPO as CustomerRepository
    participant JT as JdbcTemplate
    participant HK as HikariCP
    participant DB as H2 in-memory
    participant RM as CustomerRowMapper
    CL->>CT: GET /api/v1/customers
    CT->>REPO: findAll
    REPO->>JT: query sql rowMapper
    JT->>HK: borrow connection
    HK-->>JT: pooled Connection
    JT->>DB: execute SELECT
    DB-->>JT: ResultSet rows
    JT->>RM: mapRow rs rowNum
    RM-->>JT: Customer record
    JT->>HK: return connection to pool
    JT-->>REPO: List of Customer
    REPO-->>CT: Iterable
    CT-->>CL: 200 JSON
  
```

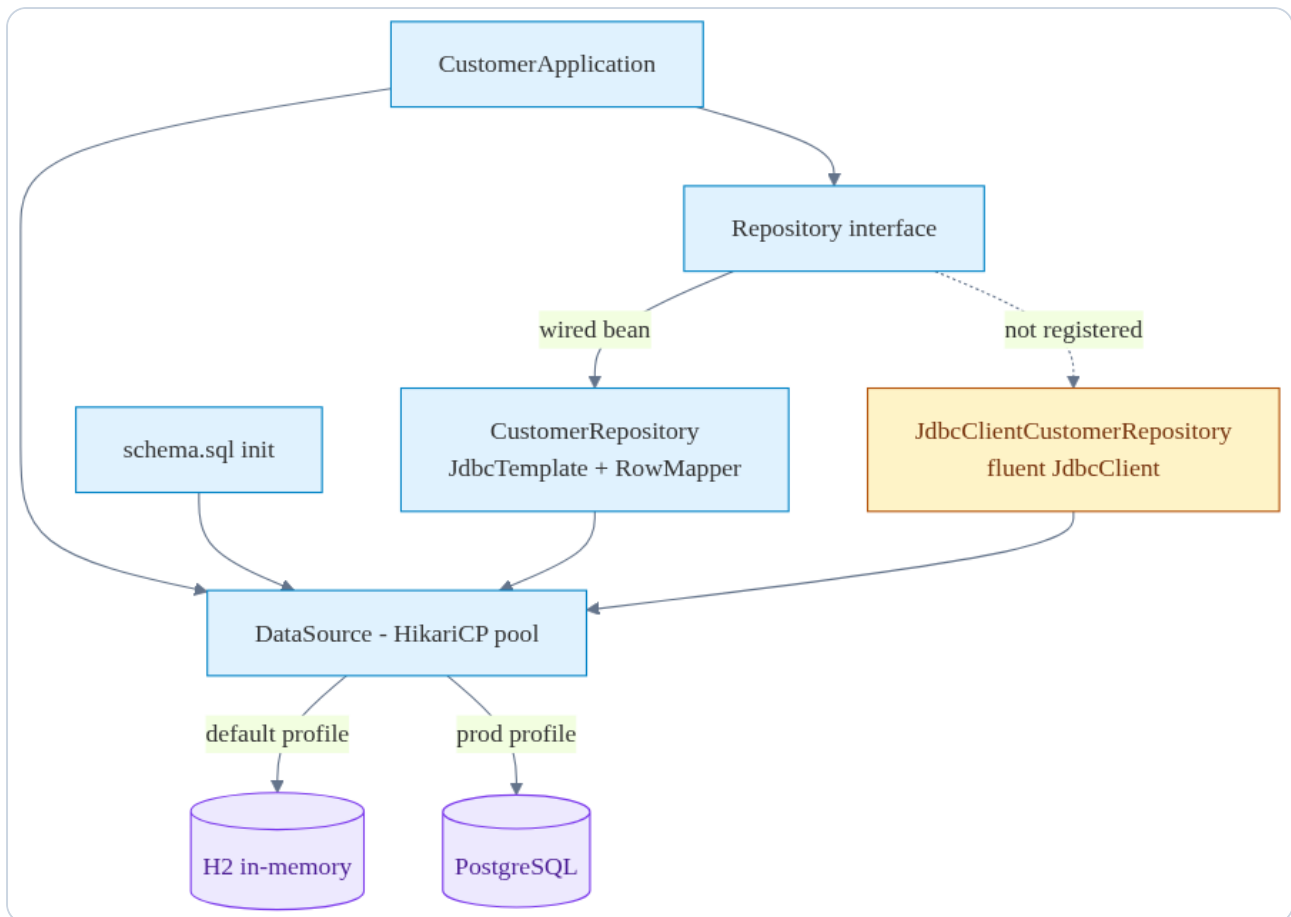


Fig 2 — Architecture & profiles. `schema.sql` and the app both feed the HikariCP `DataSource`; the default profile uses H2 and `prod` uses PostgreSQL. The `Repository` interface has the `JdbcTemplate` impl wired and the `JdbcClient` impl present but unregistered.

[Mermaid source](#)

```

%%{init: {'theme':'base','themeVariables':{'primaryColor':'#e0f2fe','primaryBorderColor':'#0284c7','line
flowchart TD
    APP["CustomerApplication"] --> DS["DataSource - HikariCP pool"]
    INIT["schema.sql init"] --> DS
    DS -->|default profile| H2["H2 in-memory"]
    DS -->|prod profile| PG["PostgreSQL"]
    APP --> IF["Repository interface"]
    IF -->|wired bean| JDBC["CustomerRepository<br/>JdbcTemplate + RowMapper"]
    IF -.->|not registered| CLNT["JdbcClientCustomerRepository<br/>fluent JdbcClient"]
    JDBC --> DS
    CLNT --> DS
    classDef store fill:#ede9fe,stroke:#7c3aed,color:#4c1d95;
    classDef alt fill:#fef3c7,stroke:#b45309,color:#78350f;
    class H2,PG store;
    class CLNT alt;
  
```

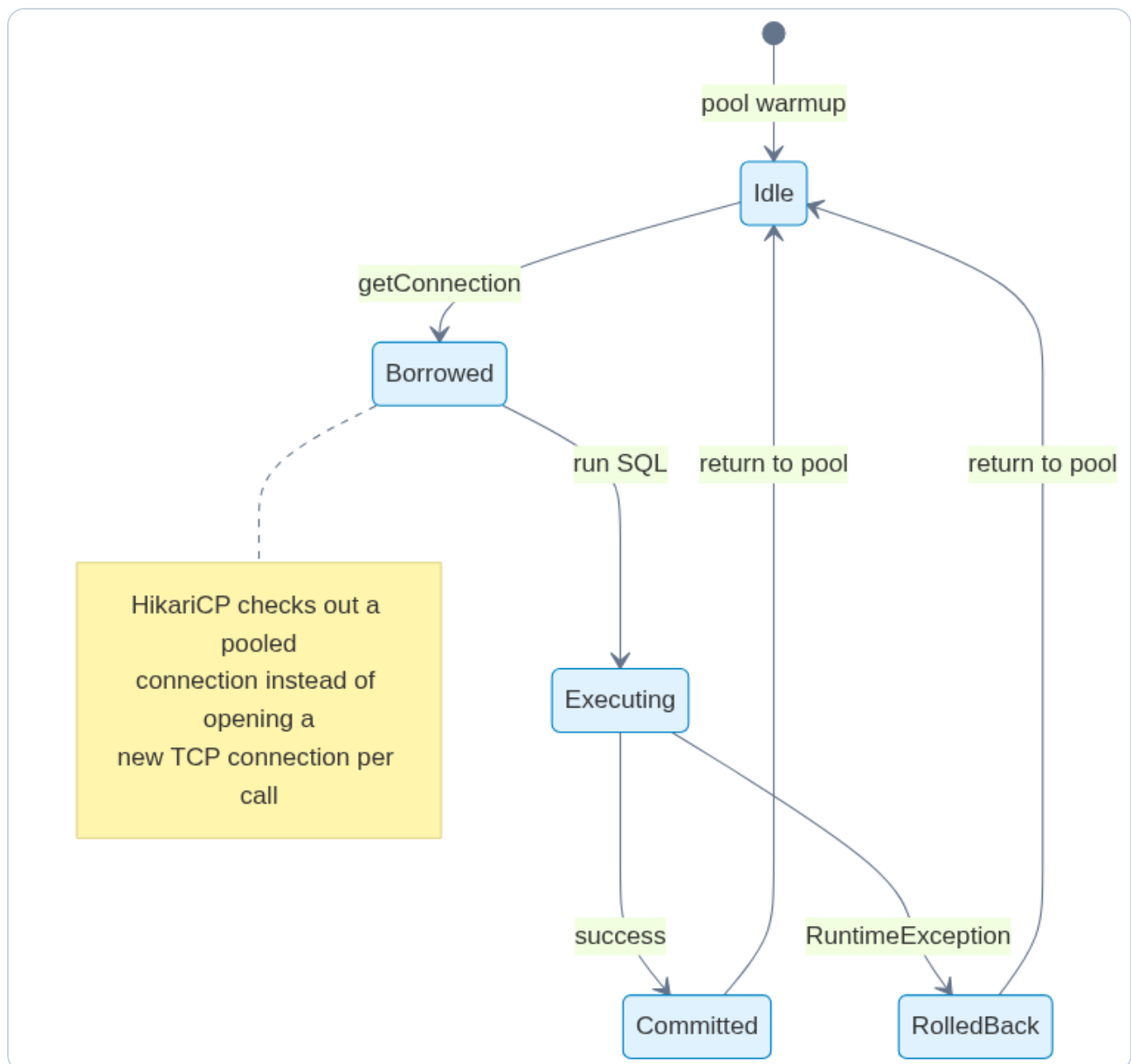


Fig 3 — Connection lifecycle. A pooled connection is checked out, used, then committed or rolled back and returned to the idle pool — no new TCP connection per request.

[Mermaid source](#)

```

%%{init: {'theme':'base','themeVariables':{'primaryColor':'#e0f2fe','primaryBorderColor':'#0284c7','line
stateDiagram-v2
    [*] --> Idle: pool warmup
    Idle --> Borrowed: getConnection
    Borrowed --> Executing: run SQL
    Executing --> Committed: success
    Executing --> RolledBack: RuntimeException
    Committed --> Idle: return to pool
    RolledBack --> Idle: return to pool
    note right of Borrowed
        HikariCP checks out a pooled
        connection instead of opening a
        new TCP connection per call
    end note

```

4 · Live run & endpoint evidence

Produced by `mvn spring-boot:run` (H2 in-memory) and `curl`.

Startup (key lines)

```

Root WebApplicationContext: initialization completed in 4889 ms
com.zaxxer.hikari.HikariDataSource      : HikariPool-1 - Starting...
com.zaxxer.hikari.pool.HikariPool      : HikariPool-1 - Added connection conn0: url=jdbc:h2:mem:testdb
com.zaxxer.hikari.HikariDataSource      : HikariPool-1 - Start completed.
Tomcat started on port 8080 (http) with context path '/'
Started CustomerApplication in 10.241 seconds (process running for 11.972)
# ApplicationReadyEvent seed (CustomerConfiguration) -> rows in H2 via JdbcTemplate:
Customer[id=1815a542-..., name=John, email=john@example.com, phone=1-800-PHONE]
Customer[id=07625777-..., name=Jane, email=jane@example.com, phone=1-800-PHONE]
Customer[id=249dbcb9-..., name=Jack, email=jack@example.com, phone=1-800-PHONE]

```

Endpoint results

Request	Result	What it demonstrates
<code>GET /api/v1/customers</code>	200	Rows read from H2 through <code>JdbcTemplate</code> + <code>RowMapper</code>
<code>POST</code> valid new email	201 + Location	<code>INSERT</code> with generated UUID
<code>POST</code> duplicate email	500	<code>UNIQUE</code> constraint → <code>DataIntegrityViolationException</code> , not mapped by the advice
<code>GET /api/v1/customers/not-a-uuid</code>	400	UUID path-variable type-conversion failure
<code>GET /h2-console</code>	404	Console enabled in properties but not reachable in this Boot 4 build

Sample responses (verbatim)

```
# POST /api/v1/customers (valid) -> 201, headers:
Location: http://localhost:8080/api/v1/customers/7abb90e7-3e16-4fce-a07b-554b8a705d7c
{"id":"7abb90e7-...", "name":"Zoe", "email":"zoe@example.com", "phone":"1-800-9999"}

# POST /api/v1/customers (duplicate email john@example.com) -> 500
{"timestamp":"2026-09-25T01:52:34.744Z", "status":500, "error":"Internal Server Error", "path":"/api/v1/customers"}

# GET /api/v1/customers/not-a-uuid -> 400
{"timestamp":"...", "status":400, "error":"Bad Request", "path":"/api/v1/customers/not-a-uuid"}
```

Run against PostgreSQL (prod profile)

The same code was re-run with `--spring.profiles.active=prod`, pointing at a real **PostgreSQL 18.2** server via an isolated local cluster (fresh `PGDATA`, a non-root service user, database `crm`, role `crm_user`). **No application code changed** — only the `DataSource` — which is the point of the profile-switch design.

```
The following 1 profile is active: "prod"
com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@43
Tomcat started on port 8080 (http) with context path '/'
Started CustomerApplication in 9.947 seconds
# ApplicationReadyEvent seeds -> rows persisted in PostgreSQL
```

`schema.sql` ran on startup; the table and constraints live in Postgres:

```
Table "public.customer"
 id      | uuid          | not null
 name    | character varying(255) | not null
 email   | character varying(255) | not null
 phone   | character varying(50)  |
Indexes:
 "customer_pkey" PRIMARY KEY, btree (id)
 "customer_email_key" UNIQUE CONSTRAINT, btree (email)
```

Request	Result	Postgres proof
<code>GET /api/v1/customers</code>	200	3 seeded rows read from PostgreSQL
<code>POST</code> new "Priya"	201	row present in <code>customer</code> (4 rows via <code>psql</code>)
<code>POST</code> duplicate email	500	<code>customer_email_key</code> UNIQUE enforced by Postgres

```
# proof via psql
SELECT id, name, email FROM customer ORDER BY name;
... | Jack   | jack@example.com
... | Jane   | jane@example.com
... | John   | john@example.com
... | Priya  | priya@example.com
(4 rows)
```

H2 vs PostgreSQL: identical application code; the active Spring profile alone swaps the `DataSource` (H2 in-memory by default → PostgreSQL under `prod`), and `schema.sql` initializes the structure in both.

5 · Session context

This document is the record of the working session that produced it.

Request

“explain code, run app and create mermaid diagrams in html file. session context should be captured in same html file” – for `chapters/chapter-04-jdbc/`.

Steps performed

1. **Explored** the module: `README.md`, `TECHNICAL_DEEP_DIVE.md`, and 9 main source files + `application.properties` / `-prod` / `schema.sql`.
2. **Explained** the code (Section 2) with file/line references.
3. **Ran** the app (`mvn spring-boot:run`, started in 10.2s) and exercised endpoints with `curl` (Section 4).
4. **Re-ran against real PostgreSQL 18.2** using an isolated local cluster and `--spring.profiles.active=prod` – verified `schema.sql` init, seeded rows, and the `UNIQUE` constraint (Section 4).
5. **Generated** three Mermaid diagrams (data-access sequence, architecture/profiles, connection-lifecycle state), rendered to PNG and embedded as data URIs.
6. **Authored** this self-contained HTML report.

Observations & suggested improvements

- **Unwired `JdbcClient` repo:** `JdbcClientCustomerRepository` is never registered as a bean, so the fluent API is only exercised in tests. To use it, add a `@Bean` (and qualify the injection point, since two `Repository` beans would then compete).
- **No transaction boundary:** the deep-dive text cites `@Transactional(rollbackFor = Exception.class)`, but the code has none; `save()` does read-then-write non-atomically. Annotating the service/repository would make the upsert atomic.
- **DB errors leak as 500:** the `UNIQUE` email violation returns Spring's default error, not RFC 7807. Add `@ExceptionHandler(DataIntegrityViolationException.class)` returning `409 Conflict` `ProblemDetail`.
- **H2 console 404:** despite `spring.h2.console.enabled=true`, `/h2-console` is not served in this run – worth verifying the console autoconfiguration on Spring Boot 4.
- **Doc/code mismatch:** the Chapter 4 deep-dive describes `maximum-pool-size: 10` etc., but no such Hikari tuning is present in `application.properties` (defaults apply).

Environment & tooling

- Runtime: Java 25 (targets Java 21); Spring Boot 4.0.6 / Spring 7.0.7; Tomcat 11; HikariCP; H2 in-memory (default) and PostgreSQL 18.2 (isolated local cluster, `prod` profile); port 8080.
- Diagrams rendered via `mermaid.link` and embedded as base64 data URIs – the HTML has **no external scripts, CSS, or image files**.
- Sources use improved Mermaid conventions: theme `%%{init}%%`, quoted labels, `box` grouping, `classDef` color-coding, and a `stateDiagram-v2`.

Reproduce

```
cd chapters/chapter-04-jdbc/customer
mvn spring-boot:run                                # default profile -> H2 in-memory
mvn spring-boot:run -Dspring-boot.run.profiles=prod # prod profile -> PostgreSQL
# then:
curl http://localhost:8080/api/v1/customers
curl -X POST http://localhost:8080/api/v1/customers \
  -H "Content-Type: application/json" \
  -d '{"name":"Dup","email":"john@example.com","phone":"1-800-0000"}' # -> 500 (UNIQUE)
```

Chapter takeaways. Spring Boot turns a URL + driver into a pooled `DataSource` automatically; `JdbcTemplate` gives explicit control (with a `RowMapper`) while `JdbcClient` offers a fluent, named-parameter style over the same pool; `schema.sql` seeds structure; and profiles swap H2 for PostgreSQL without code changes. Remember to wrap multi-statement work in `@Transactional` and to map data-integrity errors to a proper HTTP contract.

Generated from `chapters/chapter-04-jdbc/` · source at `.../customer/src/main/java/com/apress/crm/customer/`. Self-contained (no external resources). Companion to the Chapter 2 & Chapter 3 reports.